# Final Exam Study Notes

## 1. How to Clone a GitHub Repository

Cloning a repository creates a local copy of a remote GitHub repository on your machine.

**Syntax:**

```
git clone <repository-url>
```

**Examples:**

```
# Clone via HTTPS
git clone https://github.com/username/repository-name.git

# Clone into a specific directory name
git clone https://github.com/username/repository-name.git my-folder

# Clone a specific branch
git clone -b main https://github.com/username/repository-name.git
```

After cloning, navigate into the project folder:

```
cd repository-name
```

## 2. How to Use Git Commands

Git is a version control system. Below are the most commonly used commands:

| Command | Description |
|---|---|
| `git init` | Initialize a new local repository |
| `git clone <url>` | Clone a remote repository |
| `git status` | Show the state of the working directory |
| `git add <file>` | Stage a file for commit |
| `git add .` | Stage all changed files |
| `git commit -m "message"` | Commit staged changes with a message |
| `git push` | Push commits to the remote repository |

| Command | Description |
|---|---|
| `git pull` | Fetch and merge changes from remote |
| `git log` | View commit history |

**Typical Workflow:**

```
git status                    # Check what's changed
git add .                     # Stage all changes
git commit -m "Add notes"     # Commit with a message
git push origin main          # Push to GitHub
```

---

## 3. How to Write a Markdown File with Images and Proper Formatting

Markdown uses plain text syntax to add formatting.

### Headings

```
# Heading 1
## Heading 2
### Heading 3
```

### Bold and Italic

```
**Bold text**
*Italic text*
***Bold and italic***
```

### Code Blocks

Use triple backticks for multi-line code blocks:

````
```bash
echo "Hello World"
```
````

Use single backticks for inline code: `` `command` ``

### Lists

```
- Item one
- Item two
  - Nested item

1. First
2. Second
3. Third
```

### Images

```
![Alt text](path/to/image.png)
![Alt text](https://example.com/image.png)
```

### Links

```
[Link text](https://example.com)
```

### Tables

```
| Column 1 | Column 2 |
|----------|----------|
| Value A  | Value B  |
```

---

## 4. How to Convert a Markdown File to PDF

The most common tool is `pandoc`.

**Install pandoc:**

```
sudo apt install pandoc
```

**Convert:**

```
pandoc finalNotes.md -o finalNotes.pdf
```

**With a PDF engine (recommended for better formatting):**

```
sudo apt install texlive-latex-base
pandoc finalNotes.md -o finalNotes.pdf --pdf-engine=pdflatex
```

---

## 5. How to Compress (Zip) a Directory in Debian

Use the `zip` command to compress files and directories.

**Install zip (if not installed):**

```
sudo apt install zip
```

**Syntax:**

```
zip -r output.zip directory/
```

**Examples:**

```
# Zip a folder called "FinalExamStudyNotes"
zip -r FinalExamStudyNotes.zip FinalExamStudyNotes/

# Zip multiple files
zip archive.zip file1.txt file2.txt file3.txt

# Unzip a file
unzip FinalExamStudyNotes.zip
```

The `-r` flag means **recursive** — it includes all files and subdirectories inside the folder.

---

## 6. Absolute Paths vs. Relative Paths

### Absolute Path

An **absolute path** specifies the full path from the root directory `/`. It always starts with `/`.

**Examples:**

```
# Create a file using an absolute path
touch /home/student/Documents/notes.txt

# List files using an absolute path
ls /etc/apt/
```

```
# Navigate to a directory using an absolute path
cd /home/student/Downloads
```

### Relative Path

A **relative path** is based on your **current working directory**. It does not start with `/`.

**Examples:**

```
# If you are in /home/student, create a file in Documents
touch Documents/notes.txt

# Navigate up one level
cd ..

# Navigate into a subdirectory
cd FinalExamStudyNotes/
```

**Key difference:** Absolute paths work from anywhere; relative paths depend on where you currently are.

---

## 7. How to Work with the Manual Pages (man command)

The `man` command displays the manual (documentation) for a given command.

**Syntax:**

```
man <command>
```

**Examples:**

```
# Open the manual for ls
man ls

# Open the manual for grep
man grep

# Open the manual for a specific section (section 5 = file formats)
man 5 passwd
```

**Navigating inside a man page:**

| Key | Action |
|---|---|
| `Space` | Scroll down one page |

| Key | Action |
|---|---|
| b | Scroll up one page |
| q | Quit the man page |
| G | Jump to the end |
| g | Jump to the beginning |

---

## 8. How to Parse (Search) for Specific Words in the Manual Page

You can search within a man page using / (forward search).

**While inside a man page:**

```
/keyword        Search forward for "keyword"
n               Jump to next match
N               Jump to previous match
```

**Example:**

```
man ls
# Then type:
/color
# Press Enter to search, then n to find next match
```

You can also use grep to search the man page output directly:

```
man ls | grep "color"
man grep | grep "recursive"
```

---

## 9. How to Redirect Output (>, >>, and |)

### > — Overwrite Redirect

Sends output to a file, **replacing** its contents.

```
ls > filelist.txt         # Saves ls output to filelist.txt
echo "Hello" > hello.txt  # Creates hello.txt with "Hello"
```

### >> — Append Redirect

Sends output to a file, **adding** to the end without overwriting.

```
echo "Line 1" >> log.txt
echo "Line 2" >> log.txt   # log.txt now has both lines
```

### | — Pipe

Sends the output of one command as **input** to another command.

```
ls | grep ".txt"           # List only .txt files
cat file.txt | wc -l       # Count lines in a file
ps aux | grep firefox      # Find running Firefox process
```

---

## 10. How to Append Output to a File

Use >> to append (add) output to an existing file without erasing what's already there.

```
# Append a line to a file
echo "New line of text" >> myfile.txt

# Append command output to a log file
date >> log.txt

# Append ls output to an existing list
ls >> filelist.txt
```

If the file does not exist, >> will create it automatically.

---

## 11. How and When to Use Pipes ( | )

A **pipe** ( | ) takes the **standard output** of one command and feeds it as the **standard input** of another.

**Use pipes when you want to:**

- Filter output (`grep`)
- Count results (`wc`)
- Sort results (`sort`)
- Page through long output (`less`)

**Examples:**

```
# Show only .md files in current directory
ls | grep ".md"
```

```
# Count how many users are in /etc/passwd
cat /etc/passwd | wc -l

# View a long command output one page at a time
man bash | less

# Sort a file and remove duplicates
cat names.txt | sort | uniq

# Find a running process
ps aux | grep ssh
```

---

## 12. How to Use `echo` and Output Redirection to Create a File

`echo` prints text to the terminal. Combined with `>`, it can create a file with content.

```
# Create a new file with text
echo "Hello, World!" > greeting.txt

# Create a file with multiple lines using multiple echo commands
echo "Line 1" > myfile.txt
echo "Line 2" >> myfile.txt
echo "Line 3" >> myfile.txt

# Create a file with a specific message
echo "This is my final exam study notes" > README.txt
```

---

## 13. How to Use Wildcards

Wildcards are special characters that represent patterns in filenames.

| Wildcard | Meaning |
|---|---|
| `*` | Matches any number of characters |
| `?` | Matches exactly one character |
| `[abc]` | Matches any one of the listed characters |

**Copying multiple files:**

```
# Copy all .txt files to a folder
cp *.txt Documents/

# Copy all files starting with "note"
cp note* backup/
```

```
# Copy all 4-character named files
cp ???? archive/
```

**Moving multiple files:**

```
# Move all .jpg images to Pictures
mv *.jpg ~/Pictures/

# Move all files that start with "lab"
mv lab* ~/Submissions/
```

---

## 14. How to Use Brace Expansion

Brace expansion lets you create multiple files, directories, or strings in a single command.

**Syntax:**

```
command {item1,item2,item3}
```

**Examples:**

```
# Create multiple directories at once
mkdir -p project/{src,docs,tests,bin}

# Create an entire directory structure
mkdir -p school/{cis106,cis110,math101}/{notes,assignments,labs}

# Create multiple files
touch report_{january,february,march}.txt

# Create numbered directories
mkdir week{1..5}
```

Brace expansion is extremely powerful for creating organized directory structures in one line instead of running `mkdir` repeatedly.

---

## 15. How to Create a "Hello World" Shell Script

A shell script is a text file containing a sequence of commands.

**Step 1: Create the script file**

```
touch hello.sh
```

**Step 2: Open it in a text editor**

```
nano hello.sh
```

**Step 3: Write the script**

```
#!/bin/bash
# This is a simple Hello World script
echo "Hello, World!"
```

- `#!/bin/bash` — The **shebang** line. Tells the system to use bash to run the script.
- Lines starting with `#` are **comments**.

**Step 4: Make the script executable**

```
chmod +x hello.sh
```

**Step 5: Run the script**

```
./hello.sh
```

**Output:**

```
Hello, World!
```

---

## 16. How to Use Variables in a Shell Script

Variables store data that can be reused throughout a script.

**Syntax:**

```
VARIABLE_NAME="value"    # No spaces around =
echo $VARIABLE_NAME      # Access with $
```

**Example Script:**

```
#!/bin/bash
# Script demonstrating variables

NAME="Alice"
AGE=21
COURSE="CIS106"

echo "Student Name: $NAME"
echo "Age: $AGE"
echo "Course: $COURSE"
```

**Using variables in expressions:**

```
#!/bin/bash
FILE="notes.txt"
echo "Creating file: $FILE"
touch $FILE
echo "File $FILE has been created."
```

**Reading user input into a variable:**

```
#!/bin/bash
echo "Enter your name:"
read USERNAME
echo "Hello, $USERNAME!"
```

---

# 17. Command Reference

## a. awk

**Definition:** A powerful text-processing tool used to search, filter, and manipulate text files and streams, especially structured/columnar data.

**Syntax:**

```
awk 'pattern { action }' file
```

**Examples:**

```
# Print the first column of a file
awk '{ print $1 }' file.txt

# Print lines where the second field equals "Alice"
```

```
awk '$2 == "Alice" { print }' students.txt

# Print the username and home directory from /etc/passwd
awk -F: '{ print $1, $6 }' /etc/passwd

# Print the total of a column of numbers
awk '{ sum += $1 } END { print "Total:", sum }' numbers.txt

# Print lines longer than 50 characters
awk 'length($0) > 50' file.txt
```

---

### b. cat

**Definition:** Displays the contents of a file to the terminal. Also used to combine (concatenate) multiple files.

**Syntax:**

```
cat [options] file
```

**Examples:**

```
# Display contents of a file
cat notes.txt

# Display contents with line numbers
cat -n notes.txt

# Combine two files into one
cat file1.txt file2.txt > combined.txt

# Create a file with typed content (Ctrl+D to save)
cat > newfile.txt

# Display multiple files in sequence
cat header.txt body.txt footer.txt
```

---

### c. cp

**Definition:** Copies files or directories from one location to another.

**Syntax:**

```
cp [options] source destination
```

**Examples:**

```
# Copy a file to another location
cp notes.txt backup/notes.txt

# Copy and rename a file
cp notes.txt notes_backup.txt

# Copy a directory and its contents recursively
cp -r Documents/ Documents_backup/

# Copy all .txt files to a directory
cp *.txt ~/Documents/

# Copy with verbose output (shows what's being copied)
cp -v file.txt backup/
```

---

### d. `cut`

**Definition:** Extracts specific sections (columns, characters, or fields) from each line of a file or input.

**Syntax:**

```
cut [options] file
```

**Common Options:**

| Option | Description |
|---|---|
| `-d` | Specify a delimiter |
| `-f` | Select fields by number |
| `-c` | Select by character position |

**Examples:**

```
# Extract the first field from a colon-delimited file
cut -d: -f1 /etc/passwd

# Extract the username and shell from /etc/passwd
cut -d: -f1,7 /etc/passwd

# Extract characters 1 through 5 from each line
cut -c1-5 file.txt

# Extract the second column from a CSV file
cut -d, -f2 data.csv
```

```
# Use cut with a pipe to get only usernames
cat /etc/passwd | cut -d: -f1
```

---

### e. grep

**Definition:** Searches for a pattern (text or regular expression) within files and prints matching lines.

**Syntax:**

```
grep [options] pattern file
```

**Examples:**

```
# Search for the word "error" in a log file
grep "error" system.log

# Case-insensitive search
grep -i "hello" notes.txt

# Search recursively in all files in a directory
grep -r "TODO" ~/Projects/

# Show line numbers of matches
grep -n "bash" /etc/passwd

# Show lines that do NOT match
grep -v "root" /etc/passwd
```

---

### f. head

**Definition:** Displays the first lines of a file (default: first 10 lines).

**Syntax:**

```
head [options] file
```

**Examples:**

```
# Display first 10 lines of a file
head notes.txt

# Display first 5 lines
```

```
head -n 5 notes.txt

# Display first 20 lines
head -n 20 log.txt

# Display the first 100 bytes of a file
head -c 100 file.txt

# Use with a pipe to preview command output
ls -la | head -n 5
```

---

## g. `ls`

**Definition:** Lists the files and directories in a directory.

**Syntax:**

```
ls [options] [directory]
```

**Examples:**

```
# List files in current directory
ls

# Long listing format (permissions, size, date)
ls -l

# Include hidden files
ls -la

# Sort by file size
ls -lS

# List files with human-readable sizes
ls -lh ~/Documents/
```

---

## h. `man`

**Definition:** Displays the manual (help documentation) for a command.

**Syntax:**

```
man [section] command
```

**Examples:**

```
# Open the manual for ls
man ls

# Open the manual for grep
man grep

# View section 5 (file formats) for passwd
man 5 passwd

# Search for a command by keyword
man -k "copy file"

# Get a one-line summary of a command
man -f ls
```

---

### i. mkdir

**Definition:** Creates one or more new directories.

**Syntax:**

```
mkdir [options] directory_name
```

**Examples:**

```
# Create a single directory
mkdir Projects

# Create multiple directories at once
mkdir notes assignments labs

# Create nested directories (parent + child)
mkdir -p school/cis106/week1

# Create a directory structure with brace expansion
mkdir -p project/{src,docs,tests}

# Create directory and show verbose output
mkdir -v NewFolder
```

---

### j. mv

**Definition:** Moves or renames files and directories.

**Syntax:**

```
mv [options] source destination
```

**Examples:**

```
# Rename a file
mv oldname.txt newname.txt

# Move a file to a different directory
mv notes.txt ~/Documents/

# Move and rename at the same time
mv notes.txt ~/Documents/final_notes.txt

# Move all .txt files to Documents
mv *.txt ~/Documents/

# Move a directory
mv oldFolder/ ~/backup/
```

---

## k. `tac`

**Definition:** The reverse of `cat` — displays the contents of a file with lines in **reverse order** (last line first).

**Syntax:**

```
tac file
```

**Examples:**

```
# Display a file in reverse line order
tac notes.txt

# Reverse a log file to see most recent entries first
tac /var/log/syslog | head -n 20

# Reverse output and save to a new file
tac numbers.txt > reversed.txt

# Use with a pipe
cat file.txt | tac
```

---

## l. `tail`

**Definition:** Displays the last lines of a file (default: last 10 lines). Useful for monitoring log files.

**Syntax:**

```
tail [options] file
```

**Examples:**

```
# Display last 10 lines of a file
tail notes.txt

# Display last 20 lines
tail -n 20 log.txt

# Monitor a file in real time (great for logs)
tail -f /var/log/syslog

# Display last 5 lines
tail -n 5 file.txt

# Show last 50 bytes of a file
tail -c 50 file.txt
```

---

## m. `touch`

**Definition:** Creates an empty file or updates the timestamp of an existing file.

**Syntax:**

```
touch filename
```

**Examples:**

```
# Create a new empty file
touch newfile.txt

# Create multiple files at once
touch file1.txt file2.txt file3.txt

# Create files using brace expansion
touch report_{jan,feb,mar}.txt

# Update the timestamp of an existing file
```

```
touch existingfile.txt

# Create a file using an absolute path
touch /home/student/Documents/assignment.txt
```

---

## n. tr

**Definition:** Translates, deletes, or squeezes characters from standard input. Used to modify character streams.

**Syntax:**

```
tr [options] set1 [set2]
```

**Examples:**

```
# Convert lowercase to uppercase
echo "hello world" | tr 'a-z' 'A-Z'

# Convert uppercase to lowercase
echo "HELLO WORLD" | tr 'A-Z' 'a-z'

# Delete specific characters (remove all digits)
echo "abc123def456" | tr -d '0-9'

# Replace spaces with underscores
echo "hello world" | tr ' ' '_'

# Squeeze multiple spaces into one
echo "hello    world" | tr -s ' '
```

---

## o. tree

**Definition:** Displays the directory structure in a tree-like visual format. Useful for visualizing folder hierarchies.

**Syntax:**

```
tree [options] [directory]
```

**Install if needed:**

```
sudo apt install tree
```

**Examples:**

```
# Display tree of current directory
tree

# Display tree of a specific directory
tree ~/Documents/

# Show only directories (no files)
tree -d

# Limit depth to 2 levels
tree -L 2

# Show hidden files as well
tree -a ~/Projects/
```

---

*End of Final Exam Study Notes*